

TRANSACTIQ

Remaining Development Work, Productionization Roadmap & Beginner AWS Deployment Guide

A practical guide for taking the current TransactIQ project from a locally functioning and tested system toward a cloud-deployed, production-oriented engineering project.

Document date: 3 October 2026

Scope: Existing implementation + 10 remaining productionization / engineering / enhancement areas + AWS Free deployment walkthrough

Important: The 10 items are not ten missing core features. TransactIQ already has substantial payment infrastructure. These are the remaining productionization, deployment, reliability, security, performance, documentation and enhancement areas.

1. Executive Summary

The supplied TransactIQ architecture describes an institutional-style payment-processing and financial-operations platform using Laravel 11 / PHP 8.4, PostgreSQL 16, Redis 7 and React 19 + TypeScript + Vite. It already contains merchant management, payment lifecycle management, distributed idempotency, gateway simulation/failover, reliable webhooks, double-entry ledgering, settlement, reconciliation, cryptographic audit controls, risk heuristics, RBAC and an operations portal.

The same project document reports 53/53 automated feature tests passing, 376 assertions and 0 failures/errors. The next phase should therefore focus on making the system reproducible, deployable, observable, performance-tested, hardened and professionally documented.

The 10 remaining areas

#	Area	Main goal	Priority
1	Docker containerization	Run the stack consistently	High
2	CI/CD with GitHub Actions	Automatically test/build/deploy	High
3	AWS/cloud deployment	Run TransactIQ in the cloud	High
4	Monitoring & observability	Know what is happening in production	High
5	Load & performance testing	Measure concurrency and capacity	High
6	Real payment sandbox integration	Connect to a real sandbox	Medium
7	Advanced fraud / ML layer	Extend heuristics with data-driven detection	Medium
8	API documentation	Make integration easy for developers	High
9	Security hardening	Perform a production security review	High
10	Portfolio presentation	Present the engineering work professionally	High

Recommended sequence: Docker → CI/CD → AWS → Monitoring → Performance testing → API docs → Security → optional provider sandbox → optional ML → final portfolio presentation.

2. What Is Already Done

The project architecture summary supplied for TransactIQ is the baseline for this roadmap. The following are documented as implemented and should be treated as the current core unless repository testing later shows otherwise.

- Merchant multi-tenancy and customer resolution.
- Secure merchant API key handling.
- Payment transaction engine with deterministic finite-state lifecycle.
- Distributed idempotency using Redis locking and database uniqueness.
- Simulated payment gateway with deterministic scenarios and failover.
- Signed asynchronous webhooks with retries and manual replay.
- Double-entry financial ledger with debit/credit balancing.
- Settlement batching and payout lifecycle.
- CSV/JSON clearing ingestion and reconciliation.
- Cryptographic audit chaining and sensitive-data scrubbing.
- Real-time risk and velocity heuristics.
- Role-based access control.
- React operations portal and developer sandbox.

Current verification

The supplied document lists feature tests for health, schema, payment API, gateway simulation, webhooks, ledger/settlement, reconciliation and security/RBAC. It reports 53 passed tests out of 53, 376 assertions and zero failures/errors.

Baseline rule: Before starting the backlog, run the current test suite locally and save the result. Every later stage should preserve this green baseline.

3. Master Remaining-Work Checklist

Area	What you will build	Definition of done
Docker	Backend, frontend, PostgreSQL, Redis and queue-worker container	Fresh machine can start the stack from documented commands
CI/CD	GitHub Actions test/build/deploy workflows	Push triggers validation; failed tests block deployment
AWS	EC2-first deployment; RDS later if useful	Public demo works securely and reproducibly
Monitoring	Logs, health, metrics, alerts	You can detect errors, downtime and financial anomalies
Performance	Load and concurrency tests	You have measured latency, throughput and error rate
Provider sandbox	One real sandbox adapter	Success/failure/pending/webhook flows work in sandbox
ML fraud	Optional Python model/service	Model scores transactions without replacing core rules
API docs	README + OpenAPI/Postman	Another developer can integrate independently
Security	Hardening checklist and tests	Secrets, access, validation and exposure reviewed
Portfolio	Case study, screenshots, architecture and demo	Recruiter/mentor can understand the project quickly

Definition of finished

A task is finished only when it is implemented, tested, documented and reproducible. A file existing in the repository is not enough.

4. Area 1 — Docker Containerization

What it means

Docker packages an application and its environment into reproducible containers. Instead of manually installing PHP, PostgreSQL, Redis and Node on every machine, you describe the environment in configuration files.

Target architecture

```
TransactIQ
■
■■■ backend container
■ ■■■ Laravel + PHP
■■■ frontend container
■ ■■■ React/Vite build
■■■ database container
■ ■■■ PostgreSQL
■■■ redis container
■ ■■■ Redis
■■■ worker container
    ■■■ Laravel queue worker
```

Tasks

- 1 Create a backend Dockerfile.
- 2 Create a frontend Dockerfile/build stage.
- 3 Create docker-compose.yml (or Compose configuration).
- 4 Add PostgreSQL and Redis services for local development.
- 5 Add a queue-worker service using the application image.
- 6 Configure environment variables and a persistent PostgreSQL volume.
- 7 Use Compose service names rather than localhost for inter-container connections.
- 8 Run migrations and seeders inside the environment.
- 9 Verify queues and webhooks.
- 10 Document startup, shutdown, logs, migration and test commands.

```
docker compose up -d
docker compose ps
docker compose exec backend php artisan migrate
docker compose exec backend php artisan test
```

Definition of done

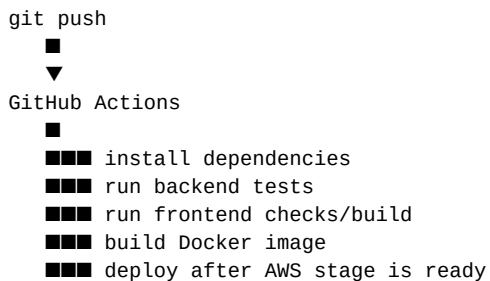
- A fresh machine with Docker can start the system.
- Laravel connects to PostgreSQL and Redis.
- Queue workers process jobs.
- Frontend communicates with backend.
- Tests pass in the intended environment.
- No production secrets are committed.

5. Area 2 — CI/CD with GitHub Actions

What it means

Continuous Integration automatically validates code changes. Continuous Delivery/Deployment packages and deploys validated changes.

Target workflow



Implementation order

- 1 Create a tests workflow first; do not deploy yet.
- 2 Install PHP and Composer dependencies.
- 3 Provide test PostgreSQL and Redis services, preferably through containers.
- 4 Run php artisan test.
- 5 Run frontend checks/build.
- 6 Add a Docker build workflow.
- 7 After those are stable, add AWS deployment.
- 8 Store deployment credentials as GitHub secrets rather than source code.

Definition of done

- Every push has a visible workflow result.
- A failed test fails the workflow.
- Docker images can be built automatically.
- Secrets are not in the repository.
- Deployment happens only after validation succeeds.

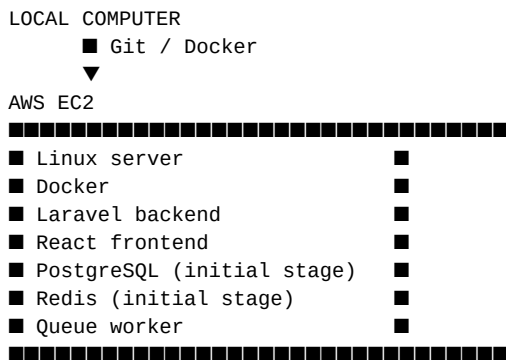
6. Area 3 — AWS Deployment

What “free” means in 2026

AWS currently states that new Free plan customers receive \$100 in credits at signup and can earn up to another \$100 through eligible activities. AWS says the Free plan lasts up to six months or until credits are exhausted, whichever happens first. The Free plan is therefore a credit-backed learning period, not a promise that AWS infrastructure is permanently free.

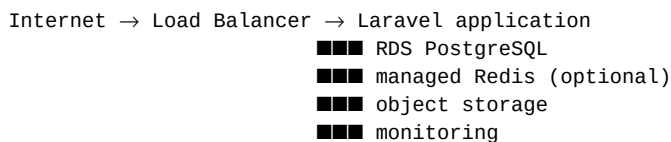
AWS also has service-specific Always Free and other free-usage offers. Limits and eligibility vary. For TransactIQ, always select resources explicitly marked Free Tier eligible in the console and monitor your credit/usage dashboard.

Beginner architecture



Why start with one EC2 server? It minimizes the number of AWS services while teaching EC2, Linux, SSH, Docker, networking, security groups, environment variables and deployment.

Later architecture



The later architecture is closer to a scalable production design but can consume credits. Introduce it after the simple deployment works.

7. AWS Free Account Setup — Step 1

Goal

Create or verify the AWS account and put cost controls in place before launching TransactIQ resources.

What you need

- A working email address you control.
- A phone number for verification.
- A valid payment method if AWS requests one during registration/verification.
- A current web browser.
- A secure place to store recovery/account information.

Security warning: Never send your AWS root password, private SSH key, secret access keys, database password or API secrets to anyone or paste them into chat.

Step 1A — Check your account

- 1 Open the official AWS Free Tier page.
- 2 Sign in if you already have an AWS account.
- 3 Find the Free Tier / Explore AWS information in the console.
- 4 Check your account plan, remaining credits and remaining time.
- 5 If you are a new customer without an account, choose Create a Free Account.

Step 1B — Create the account

- 1 Open the official AWS Free Tier page.
- 2 Choose Create a Free Account.
- 3 Complete registration and AWS verification.
- 4 If offered an account-plan choice, choose the Free plan for this learning project.
- 5 Complete the verification AWS requests.
- 6 Sign in to the AWS Management Console.

Step 1C — Set up cost protection

- 1 Open Billing and Cost Management.
- 2 Open AWS Budgets.
- 3 Create a monthly cost budget using the budget wizard/template.
- 4 Add your email for notifications.
- 5 Keep the budget active for the whole project.

AWS currently lists setting up a cost budget as one of the activities through which new customers can earn additional credits. AWS also provides a beginner cost-control tutorial for the current Free Tier.

Step 1D — Record your baseline

Write down or screenshot your plan, remaining credits, credit expiry, chosen region and budget alert. Do not share sensitive account information.

Step 1 is complete when

- Account is active.
- You know whether you are on the Free plan.
- You can see credits/eligibility.
- Budget/cost alert is configured.
- You have not launched unnecessary resources.

Stop here after Step 1. The next step is EC2. Do not create random EC2/RDS/Redis resources before confirming your actual account status and selecting the correct eligible configuration.

8. AWS Step 2 — Launch a Free-Tier-Eligible EC2 Server

What EC2 means

Amazon EC2 is a virtual computer in AWS. For TransactIQ it can initially be your cloud server.

Important Free Tier rule

AWS documentation says current Free plan users should select an instance type marked Free Tier eligible. Exact eligible types can change, so use the console's current eligibility label instead of relying on an old tutorial.

Beginner steps

- 1 Open the EC2 console.
- 2 Choose your region.
- 3 Choose Launch instance.
- 4 Name it transactiq-server.
- 5 Choose a Linux image appropriate for your account and marked eligible where applicable.
- 6 Select an instance type explicitly marked Free Tier eligible.
- 7 Create a key pair and securely store the private key.
- 8 Create a security group. Restrict SSH to your own IP where practical; allow HTTP/HTTPS when needed.
- 9 Review storage and choose the eligible/free configuration shown by AWS.
- 10 Launch the instance.
- 11 Wait for running state and health checks.
- 12 Record the public IP/DNS name.

Do not open SSH to the whole internet unless you understand the risk. AWS warns that 0.0.0.0/0 permits connections from anywhere and is unsafe for production.

Connect

AWS displays the exact connection command and username for the selected Linux image. Keep the private key on your computer.

```
ssh -i your-key.pem <username>@<public-ip>
```

Definition of done

- EC2 is running.
- You can connect securely.
- You know the region, IP and security group.
- The selected instance is Free Tier eligible for your account.
- No unnecessary resources exist.

9. AWS Step 3 — Prepare EC2 and Deploy TransactIQ

Goal

Turn the empty Linux server into a Docker host and run the same containerized TransactIQ stack that works locally.

Tasks

- 1 Update OS packages.
- 2 Install Docker using the official instructions for the selected Linux distribution.
- 3 Install/verify Docker Compose support.
- 4 Verify Docker.
- 5 Clone the TransactIQ repository.
- 6 Create production environment configuration without committing secrets.
- 7 Build and start containers.
- 8 Run Laravel migrations.
- 9 Verify application key and configuration.
- 10 Verify PostgreSQL and Redis connectivity.
- 11 Verify queue workers.
- 12 Test the health endpoint.

```
docker --version
docker compose version
```

```
git clone <your-repository>
cd transactiq
```

```
docker compose build
docker compose up -d
docker compose ps
```

```
docker compose exec backend php artisan migrate
docker compose exec backend php artisan test
```

What not to do

- Do not put .env secrets into GitHub.
- Do not expose PostgreSQL publicly.
- Do not expose Redis publicly.
- Do not use development passwords on the public server.
- Do not assume local tests automatically prove the cloud environment is correct.

10. AWS Step 4 — Optional RDS PostgreSQL Migration

Why RDS?

RDS is AWS's managed relational database service. It can replace the PostgreSQL container after your first EC2 deployment is stable.



AWS documentation currently lists PostgreSQL among RDS Free Tier offerings and identifies eligible micro classes. Exact limits depend on the current account terms, so select only configurations the AWS console marks eligible.

Migration tasks

- 1 Create RDS PostgreSQL using an eligible configuration.
- 2 Keep EC2 and RDS in the same region.
- 3 Prefer a private database configuration.
- 4 Create a database security group that allows PostgreSQL traffic only from the EC2 application security group.
- 5 Create the database and wait until available.
- 6 Copy the RDS endpoint.
- 7 Update Laravel database environment variables securely.
- 8 Run migrations and tests.
- 9 Verify payments, ledger, reconciliation, settlement and queues.
- 10 Back up before destructive changes.

Do this after the one-server deployment works. The purpose is to learn managed databases, not to complicate the first deployment.

11. Area 4 — Monitoring & Observability

Monitoring tells you whether the system is healthy. Observability helps explain why it is unhealthy.

Category	Examples
Availability	Health endpoint, uptime, container status
Application	HTTP errors, latency, exceptions
Payments	success/failure/pending volume, gateway failures
Webhooks	delivery failures, retries, replay activity
Ledger	debit/credit integrity and posting failures
Settlement	batch failures and payout status
Reconciliation	open exceptions and mismatches
Risk	blocked/reviewed transaction counts
Infrastructure	CPU, memory, disk and network

Implementation

- 1 Standardize Laravel logs.
- 2 Include safe transaction/correlation references.
- 3 Monitor EC2 and container health.
- 4 Monitor PostgreSQL and Redis.
- 5 Add application metrics.
- 6 Create alerts for critical failures.
- 7 Alert on ledger integrity failure.
- 8 Add dashboards after basic alerts work.

12. Area 5 — Load & Performance Testing

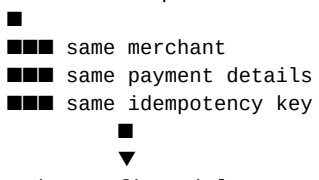
Functional tests answer whether software behaves correctly. Load tests answer how it behaves under concurrent work.

Scenarios

- 100 simultaneous payment requests.
- Many requests using one idempotency key.
- Large webhook queues.
- Concurrent ledger posting.
- Concurrent settlement generation.
- Large reconciliation files.
- Repeated API requests under rate limits.

Idempotency stress test

100 concurrent requests



Expected: one financial transaction
+ replayed duplicate responses

Metrics

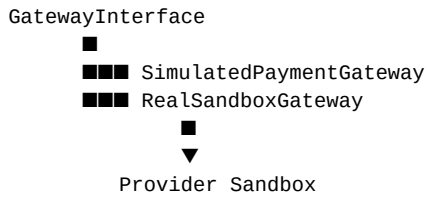
- Requests/second.
- Average latency.
- p50/p95/p99 latency.
- HTTP error rate.
- Database errors.
- Queue processing time.
- CPU and memory usage.

Suggested tool

Start with k6. Increase concurrency gradually. Load-test only your simulator or an authorized sandbox, never an uncontrolled third-party production payment endpoint.

13. Area 6 — Real Payment Provider Sandbox

The current project already has a simulated gateway and a gateway abstraction. A real sandbox adapter can be added without replacing the core transaction engine.



Tasks

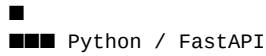
- 1 Choose one provider with a documented sandbox.
- 2 Study authentication and webhook documentation.
- 3 Implement an adapter behind the existing gateway contract.
- 4 Map provider statuses to internal TransactIQ states.
- 5 Verify webhook signatures.
- 6 Implement timeout/retry rules.
- 7 Test success, failure, pending, timeout and webhook flows.
- 8 Add integration tests that can be disabled when the sandbox is unavailable.

This is an enhancement, not a prerequisite for the core TransactIQ deployment.

14. Area 7 — Advanced Fraud Detection / ML

TransactIQ already documents heuristic risk checks for velocity, stolen-card fingerprints, repeated declines and unusually large transactions. The remaining work is an optional data-driven layer.

Payment → Rules → Rule Score → Optional ML Service → Combined Decision



Implementation

- 1 Use synthetic or authorized/anonymized transaction data.
- 2 Define features such as amount, frequency, time, history and decline patterns.
- 3 Build a baseline Python model.
- 4 Evaluate precision, recall and false positives.
- 5 Expose the model through an API only after offline evaluation.
- 6 Integrate it with the risk workflow.
- 7 Keep deterministic safety rules available for hard blocks.
- 8 Log model version and decision metadata.

Do not train a portfolio project using real cardholder data.

15. Area 8 — API Documentation & Developer Experience

The current API already covers payments, webhooks, ledger, settlement, reconciliation and security. The remaining job is to make it easy for an external developer to understand and use.

Documentation package

- README.md
- Architecture diagram
- Environment setup
- API reference
- OpenAPI specification
- Postman collection
- Authentication guide
- Webhook guide
- Idempotency guide
- Error-code reference
- Example integration

Every endpoint should explain

- Purpose and HTTP method.
- URL and authentication.
- Required headers.
- Request body.
- Success response.
- Error responses.
- Idempotency rules where applicable.
- Example curl request.
- Example response.

Developer journey

1. Obtain merchant API key
2. POST /api/v1/payments
3. Receive transaction reference
4. GET /api/v1/payments/{ref}
5. Receive signed webhook
6. Inspect settlement
7. Reconcile clearing data

16. Area 9 — Security Hardening

Existing foundations

- API key hashing/verification.
- RBAC.
- HMAC webhook signatures.
- Replay protection.
- Sensitive-data scrubbing.
- Cryptographic audit chaining.
- Risk controls.

Remaining review

Area	Questions
Secrets	Are credentials absent from Git and stored securely?
Authentication	Can unauthenticated users reach protected endpoints?
Authorization	Can Merchant A access Merchant B? Can auditors modify financial records?
Validation	Can malformed amounts/IDs bypass rules?
Injection	Are queries and external calls safely handled?
Rate limiting	Can one client overwhelm payment/webhook endpoints?
Network	Are PostgreSQL and Redis private? Is SSH restricted?
Webhooks	Are signatures, timestamps and replay rules enforced?
Logging	Are card data, secrets and unnecessary PII excluded?
Dependencies	Are vulnerable packages identified and updated?

Definition of done

You should have a security checklist, tests for important authorization boundaries, secure secret handling, restricted network exposure and documented security decisions.

17. Area 10 — Final Portfolio / Production Presentation

Recommended repository structure

```
transactiq/  
■■■ backend/  
■■■ frontend/  
■■■ docs/  
■■■ tests/  
■■■ docker/  
■■■ .github/workflows/  
■■■ docker-compose.yml  
■■■ README.md  
■■■ LICENSE
```

README structure

- 1 Project overview.
- 2 Background problem and problem statement.
- 3 How TransactIQ solves it.
- 4 Architecture.
- 5 Technology stack.
- 6 Core features.
- 7 Security design.
- 8 API documentation.
- 9 Testing results.
- 10 Docker setup.
- 11 AWS deployment.
- 12 Performance results.
- 13 Screenshots/demo.
- 14 Future roadmap.

Your project story

The strongest presentation is a technical case study: you identified reliability and financial-integrity problems in digital payments; designed the transaction lifecycle; added idempotency, failure handling, webhooks, accounting, reconciliation, settlement, audit security and risk controls; tested them; and then productionized the system with Docker, CI/CD, cloud deployment, monitoring and performance testing.

18. Complete AWS Deployment Roadmap

STEP 1 AWS account + Free plan + budget
↓
STEP 2 Free-Tier-eligible EC2
↓
STEP 3 Linux + Docker + Git
↓
STEP 4 Deploy TransactIQ containers
↓
STEP 5 Public HTTP/HTTPS access
↓
STEP 6 Verify DB + Redis + queues
↓
STEP 7 Optional RDS PostgreSQL
↓
STEP 8 Monitoring / logs / alerts
↓
STEP 9 GitHub Actions CI/CD
↓
STEP 10 Performance + security tests
↓
FINAL Document the deployed system

Stage	Skill
1	AWS account, Free Tier and billing awareness
2	EC2, Linux, key pairs and security groups
3	Docker and environment configuration
4	Deployment and troubleshooting
5	HTTP/HTTPS and web-server configuration
6	Application/database/cache connectivity
7	Managed PostgreSQL and private networking
8	Observability
9	DevOps / CI/CD
10	Reliability and security engineering

19. Beginner Glossary

Term	Simple meaning
AWS	Cloud platform for computing, storage, databases and other infrastructure.
EC2	A virtual computer/server in AWS.
RDS	Managed relational database service, including PostgreSQL.
Redis	Fast in-memory data store used for locks, caching, queues and related workloads.
Docker	Tooling for packaging software and its environment into reproducible containers.
Container	An isolated process environment containing application dependencies.
VPC	A virtual private network in AWS.
Security group	A virtual firewall controlling traffic to a resource.
SSH	Secure remote access to a Linux server.
CI/CD	Automation that tests, builds and eventually deploys code.
RDS endpoint	Network address used by the application to reach an RDS database.
Free Tier / Free plan	AWS free-usage/credit offers under specified conditions; not all are permanent.
Credit	AWS balance that offsets eligible charges during the applicable period.
Monitoring	Watching health, usage and errors.
Observability	Logs/metrics/traces that help explain system behaviour.

20. Beginner Troubleshooting Guide

Cannot connect to EC2

- Confirm instance is running.
- Use the correct private key.
- Use the username AWS shows for the selected image.
- Check the SSH security-group rule.
- Confirm public IP/DNS.
- If restricted to your IP, check whether your IP changed.

Laravel cannot connect to PostgreSQL

- Inside Docker, do not use localhost for another container; use the service name.
- Check DB_HOST/PORT/DATABASE/USERNAME/PASSWORD.
- Check PostgreSQL is running.
- Check RDS security group if using RDS.
- Reload Laravel configuration after environment changes.

Laravel cannot connect to Redis

- Check Redis is running.
- Use the Compose service name if Redis is a container.
- Check port/credentials.
- Inspect queue/cache configuration.

Works locally but not on AWS

- Check environment variables.
- Check database connectivity.
- Check storage permissions.
- Check queue worker.
- Check web server configuration.
- Check security-group rules.
- Inspect container and Laravel logs.

Worried about AWS charges

- Stop creating resources and inspect current resources.
- Check Free Tier/credits.
- Check AWS Budgets alerts.
- Confirm each resource is eligible for your account.
- Terminate resources you no longer need.
- Do not trust an old tutorial's claim that a resource is free.

21. Your Immediate Action Plan

Do these in order

- 1 Run the current TransactIQ tests locally and record the baseline.
- 2 Clean/organize the GitHub repository.
- 3 Complete Docker containerization.
- 4 Confirm the full project works through Docker locally.
- 5 Create GitHub Actions tests.
- 6 Create or verify the AWS Free account and credits.
- 7 Set up AWS Budget/cost monitoring before launching infrastructure.
- 8 Launch one Free-Tier-eligible EC2 instance.
- 9 Connect to EC2 securely.
- 10 Install Docker and deploy the containerized TransactIQ stack.
- 11 Verify the health endpoint and critical workflows.
- 12 Only then consider RDS.
- 13 Add monitoring and logs.
- 14 Add CI/CD deployment.
- 15 Run performance tests.
- 16 Perform the security review.
- 17 Write API documentation.
- 18 Create the final portfolio case study.

What you should do now

Start with AWS Step 1 only: create/sign in to your AWS account, confirm the Free plan/credits and configure the budget/cost alert. After you have done that, we can proceed to EC2 and deploy the actual TransactIQ project step-by-step.

Security reminder: When reporting back, do not send your AWS password, root credentials, private key, secret access keys, database passwords or payment details. A screenshot of the non-sensitive console status is enough.

Official AWS references used for this guide

AWS Free Tier — <https://aws.amazon.com/free/>

Free Tier credit activities —

<https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans-activities.html>

AWS cost-control tutorial — <https://docs.aws.amazon.com/hands-on/latest/control-your-costs-free-tier-budgets/control-your-costs-free-tier-budgets.html>

EC2 getting started — https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html

EC2 Free Tier tracking — <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-free-tier-usage.html>

RDS documentation — <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>

EC2-to-RDS tutorial —

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/tutorial-connect-ec2-instance-to-rds-database.html>

Source note: TransactIQ implementation status and feature inventory are based on the project architecture/examination summary supplied for this project. AWS Free plan, credits and service information were checked against AWS documentation on 3 October 2026.